

Support HTML modulaire

Master 2

6 jours

3 TP progressifs

Java / Spring Boot

Jour 1 — Fondamentaux de l'architecture logicielle

Objectifs pédagogiques

Poser les bases : architecture, qualités logicielles, SOLID, couplage, cohésion, couches et premiers arbitrages.

Temps estimé

≈ 6h incluant démos et quiz

Prérequis

POO, HTTP basique, Spring Boot de base

Table des matières

- Définition
- Qualités logicielles
- Principes SOLID
- Couplage & cohésion
- Architecture en couches
- Architecture hexagonale
- Monolithe vs distribué
- Exemple Spring Boot complet
- Quiz

Qu'est-ce que l'architecture logicielle ?

L'architecture logicielle est l'ensemble des **décisions qui structurent durablement un système** : découpage en composants, leurs responsabilités, leurs interactions, les flux de données, les contraintes non fonctionnelles et les règles d'évolution.

Les trois niveaux

Niveau	Question	Exemple
Architecture	Comment on organise le système ?	Monolithe vs microservices, choix du broker
Design	Comment on structure un composant ?	Méthodes de <code>LoanService</code> , patterns de classes
Code	Comment on implémente ?	Algorithmes, nommage, lisibilité

Mode enfant 🐼

L'**architecte** dessine le plan de la maison et décide combien d'étages il y aura. Le **designer** choisit l'emplacement précis des meubles dans chaque pièce. Le **développeur** pose réellement les briques, une à une, en suivant le plan.

Mode pro 🎓

Une architecture réussie maximise la **vélocité à long terme** : changements métier rapides, déploiements sûrs, maintenance prévisible et maîtrise du coût total de possession (TCO). Elle répond explicitement à des *attributs de qualité* (ISO 25010).

"Making the architecture explicit is the key to making it evolvable." — Sam Newman

Qualités logicielles essentielles

Toute décision architecturale doit se justifier par au moins une qualité cible. Sinon, c'est une décision décorative ou risquée.

Qualité	Question à se poser	Exemple Campus Library
Maintenabilité	Peut-on modifier sans tout casser ?	Ajouter un nouveau type d'emprunt sans toucher au contrôleur
Scalabilité	Peut-on absorber plus de charge ?	Passer de 1 000 à 20 000 étudiants actifs
Performance	Répond-on assez vite ?	Recherche catalogue < 300 ms au p99
Disponibilité	Le service reste-t-il accessible ?	Catalogue consultable même si notifications HS
Sécurité	Accès et données protégés ?	JWT + RBAC + validation des entrées
Testabilité	Peut-on tester chaque pièce isolément ?	Domaine testable sans Spring, sans BD
Observabilité	Sait-on ce qui se passe en prod ?	Logs structurés, métriques, tracing distribué

⚠ Les qualités s'opposent parfois

Performance maximale et maintenabilité maximale sont souvent en tension. Plus on optimise pour la performance (cache, dénormalisation, code bas niveau), plus le code est difficile à modifier. **Le rôle de l'architecte est de gérer ces compromis explicitement**, pas de les ignorer.

✅ Prioriser les qualités dès le départ

En début de projet, listez vos 3 qualités prioritaires et leurs seuils mesurables. Ex : *disponibilité* ≥ 99,5%, *latence p95* < 500ms, *MTTR* < 1h. Ces seuils guideront toutes vos décisions architecturales.

Principes SOLID

Les principes SOLID (Robert C. Martin, ~2000) sont les fondations du design orienté objet. Ils s'appliquent au niveau classe, mais aussi au niveau module et service.

S — Single Responsibility Principle

"Une classe ne doit avoir qu'une seule raison de changer."

✗ Mauvais — trop de responsabilités

```
public class LoanService {
    // Règle métier
    public void createLoan(LoanRequest req) { ... }
    // Persistance – autre responsabilité
    public void saveToDatabase(Loan loan) { ... }
    // Notification – encore une autre
    public void sendEmailToStudent(Loan loan) { ... }
    // Rapport – une de plus
    public String generateMonthlyReport() { ... }
}
```

✓ Correct — une responsabilité par classe

```
// Chaque classe a une seule raison de changer
public class LoanApplicationService { /* règles métier */ }
public class LoanRepository { /* persistance */ }
public class LoanNotificationService { /* notifications */ }
public class LoanReportService { /* rapports */ }
```

O — Open/Closed Principle

"Ouvert à l'extension, fermé à la modification." Ajouter un comportement ne doit pas nécessiter de modifier le code existant.

```
// ✗ Mauvais : ajouter un type de notification oblige à modifier la méthode
public void notify(String type, Loan loan) {
    if (type.equals("EMAIL")) { /* email */ }
    else if (type.equals("SMS")) { /* SMS – ajouté plus tard, modifie la méthode */ }
}

// ✓ Correct : chaque nouveau type est une nouvelle implémentation
public interface LoanNotifier {
    void notify(Loan loan);
}

public class EmailLoanNotifier implements LoanNotifier { ... }
public class SmsLoanNotifier implements LoanNotifier { ... }
// Ajouter PushNotifier ne touche pas au code existant
```

L — Liskov Substitution Principle

"Un sous-type doit pouvoir remplacer son type parent sans casser le comportement."

```
// ❌ Violation LSP : DigitalBook ne peut pas être emprunté comme Book
public class Book {
    public void borrow() { /* décrémente le stock physique */ }
}

public class DigitalBook extends Book {
    @Override
    public void borrow() {
        throw new UnsupportedOperationException("Pas de stock physique !");
        // L'appelant qui reçoit un Book s'attend à pouvoir appeler borrow()
    }
}

// ✅ Correct : séparer les abstractions
public interface Borrowable { void borrow(); }
public class PhysicalBook implements Borrowable { ... }
public class DigitalBook { /* pas Borrowable, accès différent */ }
```

I — Interface Segregation Principle

"Ne forcez pas un client à dépendre d'interfaces qu'il n'utilise pas."

```
// ❌ Interface trop large – les clients n'ont pas besoin de tout
public interface BookRepository {
    Book findById(UUID id);
    void save(Book book);
    void delete(UUID id);
    List<Book> findAllForReport(); // utilisé seulement par le service de reporting
    List<Book> findExpiredLoans(); // utilisé seulement par le job de nettoyage
}

// ✅ Interfaces ciblées par cas d'usage
public interface BookReader { Book findById(UUID id); }
public interface BookWriter { void save(Book book); void delete(UUID id); }
public interface BookReporting { List<Book> findAllForReport(); }
```

D — Dependency Inversion Principle

"Les modules de haut niveau ne doivent pas dépendre des modules de bas niveau. Les deux doivent dépendre d'abstractions." C'est la base de l'injection de dépendances.

```
// ❌ La couche métier dépend directement de l'infra
public class LoanApplicationService {
    private final PostgresLoanRepository repo; // dépendance concrète !

    public void createLoan(LoanRequest req) {
        repo.save(/* ... */);
    }
}

// ✅ La couche métier dépend d'une abstraction (port)
public interface LoanRepository {
    void save(Loan loan);
}
```

```
Optional<Loan> findById(UUID id);
}

@Service
public class LoanApplicationService {
    private final LoanRepository loanRepository; // abstraction injectée

    public LoanApplicationService(LoanRepository loanRepository) {
        this.loanRepository = loanRepository;
    }

    // On peut maintenant injecter PostgresLoanRepository, InMemoryLoanRepository
    // (pour les tests), ou MongoLoanRepository sans changer cette classe.
}
```

Mode pro 🎓 — SOLID à l'échelle des services

SOLID ne s'applique pas qu'aux classes. Au niveau microservices : **SRP** → chaque service a un seul domaine. **OCP** → on ajoute des fonctionnalités par nouveaux services. **DIP** → les services dépendent d'interfaces (contrats d'API, schémas d'événements), pas d'implémentations concrètes.

Couplage, cohésion et séparation des responsabilités

Ces deux métriques qualitatives permettent d'évaluer la santé d'une architecture.

Forte cohésion ✓

Tous les éléments d'un composant travaillent ensemble vers **un seul but**. Changer l'un n'implique pas de changer les autres.

Exemple : `LoanService` ne contient que les règles d'emprunt.

Faible couplage ✓

Les composants ont **peu de connaissances** les uns sur les autres. On peut les changer, tester ou remplacer indépendamment.

Exemple : `LoanService` ne sait pas si le repo est Postgres ou MongoDB.

Visualiser le couplage

✗ Contrôleur trop couplé — tout dans une méthode

```
@PostMapping("/loans")
public ResponseEntity<> create(@RequestBody LoanRequest req) {
    // 1. Validation manuelle — devrait être dans Bean Validation
    if (req.getBookId() == null) throw new RuntimeException("bookId requis");

    // 2. Accès direct à la BD — devrait passer par un service
}
```

```

Book book = bookRepository.findById(req.getBookId()).orElseThrow();

// 3. Règle métier dans Le contrôleur – devrait être dans Le domaine
if (book.getStock() <= 0) throw new RuntimeException("Plus de stock");
book.setStock(book.getStock() - 1);
bookRepository.save(book);

// 4. Envoi de mail – devrait être dans un service dédié
mailService.send(req.getStudentEmail(), "Emprunt confirmé");

// 5. Mapping manuel – devrait être dans un mapper
LoanResponse resp = new LoanResponse();
resp.setId(loan.getId());
return ResponseEntity.ok(resp);
}

```

✅ Contrôleur correctement découplé

```

@PostMapping("/loans")
public ResponseEntity<LoanResponse> create(
    @Valid @RequestBody LoanRequest request) {
    // Le contrôleur délègue tout – il ne fait que recevoir et renvoyer
    LoanResponse response = loanApplicationService.createLoan(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(response);
}

```

Mode pro 🎓 — Loi de Déméter

La **Loi de Déméter** (*Principle of Least Knowledge*) dit qu'un objet ne doit parler qu'à ses *amis directs* — jamais à des amis d'amis. Un symptôme de violation : les chaînes de points `service.getRepository().findAll().stream()...`. Chaque point supplémentaire est un couplage invisible.

Architecture en couches (Layered Architecture)

L'architecture en couches est le point de départ le plus courant. Elle sépare typiquement **4 préoccupations** qui changent à des rythmes différents.

web / controller	← Reçoit HTTP, valide la forme
application	← Orchestre les cas d'usage
domain	← Règles métier pures
infrastructure	← BDD, mail, broker, API externes

Couche	Responsabilité	Technologie	Peut dépendre de
Web	Mapper HTTP ↔ objets métier	Spring MVC, @RestController	Application
Application	Orchestrer les cas d'usage	@Service	Domain, Infrastructure (via ports)
Domain	Règles métier, invariants	POJO pur, aucun framework	Personne
Infrastructure	Détails techniques	JPA, Kafka, RestTemplate	Domain (implémente les ports)

Mode enfant 🧒

La couche web c'est le **guichet** qui reçoit les clients et parle leur langage. La couche application c'est le **chef de projet** qui coordonne. Le domaine c'est l'**expert métier** qui connaît les règles. L'infrastructure c'est l'**équipe technique** qui s'occupe des outils.

⚠ Règle d'or des couches

Les dépendances ne vont que **vers le bas** (ou via des abstractions). Le domaine ne doit *jamais* importer une classe Spring, JPA, ou Kafka. Si c'est le cas, vous avez un couplage à l'infrastructure dans votre logique métier — le pire des couplages.

✅ Vérifier les dépendances avec ArchUnit

```
// Test automatique qui vérifie les règles d'architecture
@AnalyzeClasses(packages = "fr.univ.library")
public class LayerArchitectureTest {

    @ArchTest
    static final ArchRule domain_should_not_depend_on_infra =
        noClasses()
            .that().resideInAPackage("..domain..")
            .should().dependOnClassesThat()
            .resideInAPackage("..infrastructure..");

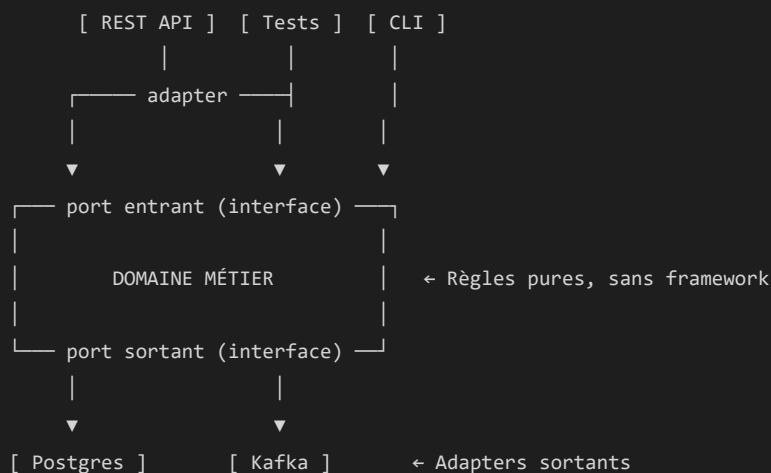
    @ArchTest
    static final ArchRule controllers_should_not_access_repos =
        noClasses()
            .that().resideInAPackage("..web..")
            .should().dependOnClassesThat()
            .resideInAPackage("..repository..");
}
```

Architecture hexagonale (Ports & Adapters)

L'architecture hexagonale (Alistair Cockburn, 2005) pousse plus loin la séparation des couches. Le **domaine est au centre**, isolé du monde extérieur par des **ports** (interfaces) et des **adapters** (implémentations).

Mode enfant 🧒

Ton téléphone (le domaine) ne sait pas comment il est chargé. Il expose juste une prise USB-C (le port). Tu peux brancher un chargeur secteur, une batterie portable ou un câble voiture (les adapters). Le téléphone n'a pas besoin de changer quand tu changes de chargeur.



Port entrant (driven by the outside world)

```

// Port : interface définie par Le DOMAINE
public interface LoanUseCase {
    LoanId createLoan(CreateLoanCommand command);
    void returnLoan(LoanId loanId);
}

// Adapter entrant : traduit HTTP → commande domaine
@RestController
@RequestMapping("/api/v1/loans")
public class LoanController {

    private final LoanUseCase loanUseCase; // dépend du PORT, pas de L'impl

    @PostMapping
    public ResponseEntity<LoanResponse> create(
        @Valid @RequestBody CreateLoanRequest req) {
        CreateLoanCommand cmd = new CreateLoanCommand(
            req.studentId(), req.bookId()
        );
        LoanId id = loanUseCase.createLoan(cmd);
        return ResponseEntity.created(URI.create("/api/v1/loans/" + id))
            .build();
    }
}
  
```

Port sortant (driven by the domain)

```

// Port : interface définie par Le DOMAINE (ce dont il a besoin)
public interface LoanRepository {
  
```



```

void save(Loan loan);
Optional<Loan> findById(LoanId id);
}

// Adapter sortant : implémentation technique (infra)
@Repository
public class JpaLoanRepository implements LoanRepository {

    private final LoanJpaEntityRepository jpa; // Spring Data JPA

    @Override
    public void save(Loan loan) {
        LoanJpaEntity entity = LoanMapper.toEntity(loan);
        jpa.save(entity);
    }

    @Override
    public Optional<Loan> findById(LoanId id) {
        return jpa.findById(id.value()).map(LoanMapper::toDomain);
    }
}

// Pour Les tests : adapter in-memory, sans BD
public class InMemoryLoanRepository implements LoanRepository {
    private final Map<LoanId, Loan> store = new HashMap<>();

    @Override public void save(Loan loan) { store.put(loan.getId(), loan); }
    @Override public Optional<Loan> findById(LoanId id) { return Optional.ofNullable(store.get(id)); }
}

```

Mode pro 🧑‍💻 — Bénéfices concrets

- **Tests ultra-rapides** : le domaine se teste avec des repositories in-memory, sans Spring, sans BD. Chaque test s'exécute en millisecondes.
- **Remplacement de technologie** : migrer de Postgres vers MongoDB = écrire un nouvel adapter, sans toucher au domaine.
- **Multi-canaux** : le même domaine est accessible par REST, CLI, batch, gRPC — chacun est un adapter différent.

Monolithe vs distribué — choisir au bon moment

Le choix n'est pas "quel est le meilleur ?" mais "quel est le plus *adapté à mon contexte* ?"

Critère	Monolithe bien structuré	Microservices
Déploiement	Simple, un seul artefact	Complexe (orchestration, CI/CD par service)
Debug	Stack trace locale, directe	Nécessite tracing distribué
Scalabilité ciblée	Tout monte ensemble	On scale uniquement le service sous charge
Latence interne	Appels mémoire (~ns)	Appels réseau (~ms, avec risques)

Critère	Monolithe bien structuré	Microservices
Autonomie d'équipe	Coordination nécessaire	Forte si bien découpé (Loi de Conway)
Taille d'équipe cible	1–10 développeurs	10+ développeurs, plusieurs équipes
Maturité du domaine	Domaine encore flou / MVP	Domaine bien compris et stabilisé

⚠ Microservices ≠ modernité

Un système distribué n'est pas "plus moderne" par nature. Il **déplace des problèmes** (couplage, état partagé) vers de *nouveaux problèmes* (réseau, cohérence, observabilité, sécurité, exploitation). Commencer par un monolithe bien structuré, découper ensuite quand la douleur est réelle.

Le Monolith-First Pattern (Martin Fowler)

La recommandation de Martin Fowler : **commencer par un monolithe**, identifier les modules qui ont des contraintes de scalabilité, de déploiement ou d'équipe différentes, puis extraire *seulement ceux-là* en services. Le monolithe en couches (voire hexagonal) facilite cette extraction car les modules sont déjà bien séparés.

Mode pro 🎓 — Le Modular Monolith

Entre monolithe classique et microservices, il existe le **Modular Monolith** : un seul processus, mais des modules internes avec des frontières strictes (pas d'accès direct aux tables d'un autre module, communication via des interfaces explicites). C'est souvent le meilleur point d'entrée avant de distribuer.

Exemple Spring Boot complet — Créer un livre

Voici comment les couches s'articulent pour un cas d'usage simple : créer un livre dans le catalogue.

1. DTO d'entrée — couche web

```
// Le DTO valide la forme (HTTP) – pas les règles métier
public record CreateBookRequest(
    @NotBlank(message = "Le titre est obligatoire")
    String title,

    @NotBlank(message = "L'auteur est obligatoire")
    String author,

    @Positive(message = "Le stock doit être positif")
    @Max(value = 1000, message = "Stock limité à 1000")
    int stock,

    @ISBN(message = "Format ISBN invalide")
    String isbn
) {}

// DTO de sortie – ne jamais exposer L'entité JPA !
```

```
public record BookResponse(  
    UUID id,  
    String title,  
    String author,  
    int stock,  
    String isbn,  
    Instant createdAt  
) {}
```

2. Contrôleur — couche web

```
...  
  
@RestController  
@RequestMapping("/api/v1/books")  
@RequiredArgsConstructor  
@Tag(name = "Catalogue", description = "Gestion des livres")  
public class BookController {  
  
    private final BookApplicationService bookService;  
  
    @PostMapping  
    @ResponseStatus(HttpStatus.CREATED)  
    @Operation(summary = "Créer un nouveau livre")  
    public BookResponse create(  
        @Valid @RequestBody CreateBookRequest request) {  
        return bookService.create(request);  
    }  
  
    @GetMapping("/{id}")  
    public BookResponse findById(@PathVariable UUID id) {  
        return bookService.findById(id);  
    }  
  
    @GetMapping  
    public Page<BookResponse> search(  
        @RequestParam(required = false) String query,  
        Pageable pageable) {  
        return bookService.search(query, pageable);  
    }  
}
```

3. Service d'application — couche application

```
...  
  
@Service  
@RequiredArgsConstructor  
@Transactional  
public class BookApplicationService {  
  
    private final BookRepository bookRepository;  
    private final BookMapper bookMapper;  
    private final BookEventPublisher eventPublisher;  
  
    public BookResponse create(CreateBookRequest request) {
```

```

// Vérification de l'unicité de l'ISBN (règle métier)
if (bookRepository.existsByIsbn(request.isbn())) {
    throw new BookAlreadyExistsException(request.isbn());
}

// Création de l'entité domaine
Book book = new Book(
    UUID.randomUUID(),
    request.title(),
    request.author(),
    request.stock(),
    request.isbn()
);

// Persistance via le port
bookRepository.save(book);

// Événement domaine (optionnel à ce stade)
eventPublisher.publish(new BookCreatedEvent(book.getId()));

return bookMapper.toResponse(book);
}

@Transactional(readOnly = true)
public BookResponse findById(UUID id) {
    return bookRepository.findById(id)
        .map(bookMapper::toResponse)
        .orElseThrow(() -> new BookNotFoundException(id));
}
}

```

4. Entité domaine — couche domaine

```

// Entité riche : porte ses invariants et ses règles métier
public class Book {
    private final UUID id;
    private String title;
    private String author;
    private int stock;
    private final String isbn;

    public Book(UUID id, String title, String author, int stock, String isbn) {
        if (stock < 0) throw new IllegalArgumentException("Stock négatif interdit");
        this.id = id;
        this.title = title;
        this.author = author;
        this.stock = stock;
        this.isbn = isbn;
    }

    // Méthode métier – encapsule la règle "un emprunt décrémente le stock"
    public void decrementStock() {
        if (this.stock <= 0) {
            throw new BookNotAvailableException(this.id);
        }
    }
}

```

```

    }
    this.stock--;
}

public void incrementStock() { this.stock++; }

// Getters (pas de setters publics — encapsulation)
public UUID getId() { return id; }
public int getStock() { return stock; }
public String getTitle() { return title; }
public String getAuthor() { return author; }
public String getIsbn() { return isbn; }
}

```

5. Gestion globale des erreurs — couche web

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BookNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ProblemDetail handleNotFound(BookNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatus(HttpStatus.NOT_FOUND);
        pd.setTitle("Livre introuvable");
        pd.setDetail(ex.getMessage());
        return pd;
    }

    @ExceptionHandler(BookNotAvailableException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public ProblemDetail handleNotAvailable(BookNotAvailableException ex) {
        ProblemDetail pd = ProblemDetail.forStatus(HttpStatus.CONFLICT);
        pd.setTitle("Livre indisponible");
        pd.setDetail("Le livre demandé n'a plus de stock disponible.");
        return pd;
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ProblemDetail handleValidation(MethodArgumentNotValidException ex) {
        ProblemDetail pd = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
        pd.setTitle("Données invalides");
        Map<String, String> errors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors()
            .forEach(fe -> errors.put(fe.getField(), fe.getDefaultMessage()));
        pd.setProperty("errors", errors);
        return pd;
    }
}

```

6. Test du domaine — sans Spring, ultra-rapide

```
class BookTest {

    @Test
    void decrementStock_should_reduce_stock_by_one() {
        Book book = new Book(UUID.randomUUID(), "Clean Code",
                               "R. Martin", 5, "978-0-13-235088-4");
        book.decrementStock();
        assertEquals(book.getStock(), 4);
    }

    @Test
    void decrementStock_should_throw_when_stock_is_zero() {
        Book book = new Book(UUID.randomUUID(), "Clean Code",
                               "R. Martin", 0, "978-0-13-235088-4");
        assertThrowsBy(book::decrementStock)
            .assertInstanceOf(BookNotAvailableException.class);
    }
}
```

✓ Règle mnémotechnique

Le contrôleur expose. Le service d'application orchestre. Le domaine décide. L'infrastructure exécute. Si vous trouvez de la logique `if/else` métier dans le contrôleur ou des annotations `@Autowired` dans le domaine, quelque chose ne va pas.

Quiz de fin

1. Pourquoi la séparation des responsabilités améliore-t-elle la maintenabilité ?

► [Voir la réponse](#)

2. Dans quel cas un monolithe reste-t-il un bon choix ?

► [Voir la réponse](#)

3. Quelle différence entre couplage fort et forte cohésion ?

► [Voir la réponse](#)

4. Quel principe SOLID est violé si le domaine importe `javax.persistence.Entity` ?

► [Voir la réponse](#)

5. Pourquoi tester le domaine sans Spring est-il une bonne pratique ?

► [Voir la réponse](#)

6. Expliquez la différence entre l'architecture en couches et l'architecture hexagonale.

► [Voir la réponse](#)

[← Introduction](#)

[Jour 2 →](#)